

Lecture notes

Resumen

El siguiente documento es una recopilación de notas he ido haciendo a lo largo del proyecto, con el fin de tener un registro de los avances y decisiones tomadas durante el desarrollo del mismo. Estas notas incluyen detalles técnicos, desafíos encontrados, soluciones implementadas y cualquier otra información relevante que pueda ser útil en el futuro. Es posible que estas notas contengan errores o información incompleta, por lo que se recomienda revisarlas con cuidado y complementarlas con otras fuentes de información si es necesario. La idea es ir actualizando este documento a medida que se avanza en el proyecto, para mantener un registro claro y organizado de todo el proceso.

Contents

1	Introducción	3
1.1	Marco Bayesiano	3
1.2	Interpretación de la likelihood y la posterior	3
1.3	Enfoques Computacionales	4
2	Approximate Bayesian Computation (ABC)	4
3	Neural Simulation-Based Inference	5
3.1	Neural Posterior Estimation (NPE)	6
3.1.1	Estimador de densidad	6
3.2	Neural Likelihood Estimation (NLE)	7
3.3	Neural Ratio Estimation (NRE)	8
4	Esquemas de compresión de datos	8
4.1	Summary Statistics	8
4.2	Information-Theoretic Compression	8
4.3	End-to-End Neural Compression	9
5	Sequential Neural Posterior Estimation	9
6	Diagnostics and tests	10
6.1	Posterior predictive checks (PPC)	10
6.2	Coverage diagnostics	10
7	Toy model	12

1 Introducción

Esta sección buscará dar una idea general de cómo se conectan los modelos teóricos con los datos reales. En el fondo, hay dos formas de ver este problema: el problema directo y el problema inverso. El problema directo es más sencillo, porque consiste en tomar parámetros Θ y generar datos x usando un modelo, como si estuviéramos simulando la realidad. En cambio, el problema inverso es más complicado, ya que parte de datos observados x_0 y trata de encontrar qué parámetros Θ los generaron. Esto implica explorar muchas posibles soluciones, lo que lo convierte en la base de la inferencia estadística.

1.1 Marco Bayesiano

La inferencia bayesiana es una forma estructurada de abordar el problema inverso. La idea principal es tratar los parámetros Θ como variables aleatorias en lugar de valores fijos, lo que permite modelar la incertidumbre de forma explícita. Se parte de un prior $p(\Theta)$, que representa lo que creemos antes de ver los datos, y de una likelihood $p(x|\Theta)$, que describe cómo el modelo genera datos. A partir de esto, se usa el teorema de Bayes:

$$p(\Theta|x) = \frac{p(x|\Theta)p(\Theta)}{p(x)} \quad (1.1)$$

donde la evidencia está dada por:

$$p(x) = \int p(x|\Theta)p(\Theta) d\Theta \quad (1.2)$$

La interpretación es bastante intuitiva: el prior representa las creencias iniciales, la likelihood mide qué tan bien el modelo explica los datos, y la posterior corresponde a las creencias actualizadas después de observar los datos. También es útil considerar la distribución conjunta $p(x, \Theta) = p(x|\Theta)p(\Theta)$, ya que desde ella se pueden obtener todas las demás distribuciones mediante marginalización o condicionamiento. En ejemplos simples, todo esto se puede calcular de forma exacta, pero en problemas reales, donde los modelos son complejos o tienen muchos parámetros, estos cálculos se vuelven complicados.

1.2 Interpretación de la likelihood y la posterior

Algo importante es cómo interpretar correctamente la likelihood y la posterior. Muchas veces se dice informalmente que ambas son distribuciones de probabilidad, pero esto no es del todo preciso sin contexto. La likelihood $p(x|\Theta)$, estrictamente hablando, es una distribución de probabilidad sobre los datos x para un valor fijo de Θ . Sin embargo, cuando se usa en inferencia, se suele ver como una función de Θ con los datos fijos. En ese sentido, no es una distribución de probabilidad sobre Θ , sino más bien una *familia de distribuciones* indexada por los parámetros. Es decir, para cada valor de Θ tenemos una distribución distinta sobre x . Por otro lado, la posterior $p(\Theta|x)$ sí es una distribución de probabilidad sobre los parámetros Θ , una vez que los datos x están fijos. Pero incluso aquí hay una sutileza: en realidad también estamos hablando de una familia de distribuciones, ya que para cada conjunto de datos distinto obtendríamos una posterior distinta. Tanto la likelihood como la posterior

dependen de qué variable se esté fijando y cuál se está considerando como variable aleatoria. Pensarlas como familias de distribuciones ayuda a evitar confusiones conceptuales.

1.3 Enfoques Computacionales

En la práctica, la distribución posterior $p(\Theta|x)$ casi nunca tiene una forma cerrada y puede tener estructuras complicadas, como múltiples modos o regiones difíciles de explorar. Por eso, es necesario recurrir a métodos computacionales para aproximarla. Los métodos clásicos como MCMC [12] o Nested Sampling [25] se basan en evaluar la likelihood $p(x_0|\Theta)$ y construir esquemas de muestreo que evitan calcular directamente la evidencia. Métodos más avanzados, como Hamiltonian Monte Carlo [3] o la Inferencia Variacional [16], además utilizan gradientes $\nabla_{\Theta} \log p(x_0|\Theta)$ para moverse de manera más eficiente en espacios de alta dimensión.

Por otro lado, existe un enfoque diferente llamado inferencia basada en simulación (SBI) [6], que no requiere evaluar la likelihood de forma explícita. En su lugar, solo necesita poder generar muestras conjuntas (x, Θ) a partir del modelo. Esto resulta especialmente útil cuando la likelihood es desconocida, muy difícil de calcular o el modelo funciona como una caja negra. Sin embargo, este enfoque implica aprender una aproximación de la distribución posterior a partir de datos simulados. En general, cada método hace distintos supuestos sobre qué se puede calcular del modelo, lo que determina cuándo es aplicable. Estas ideas vienen desarrollándose desde hace décadas, con orígenes en trabajos de los años 80, y dieron lugar a métodos como la Computación Bayesiana Aproximada (ABC).

2 Approximate Bayesian Computation (ABC)

La idea de Approximate Bayesian Computation (ABC) [18, 24] es un buen punto de partida para entender métodos modernos de inferencia basada en simulación. Básicamente, ABC es como la versión clásica (no con redes neuronales) de estos métodos. La motivación principal es que muchas veces no podemos calcular la verosimilitud $p(x|\Theta)$, pero sí podemos simular datos desde el modelo. El algoritmo funciona de la siguiente manera. Primero se muestrean parámetros desde el prior $\Theta \sim p(\Theta)$. Luego, para cada Θ , se generan datos simulados $x \sim p(x|\Theta)$ usando el modelo directo. La idea es comparar estos datos simulados con los datos observados x_0 , pero no directamente, sino usando estadísticas resumen $s(x)$ de menor dimensión. Esto se hace porque comparar datos completos puede ser muy costoso o poco informativo. Se define entonces una distancia entre datos simulados y observados como:

$$d(x, x_0) = \|s(x) - s(x_0)\| \quad (2.1)$$

Luego, se aceptan solo aquellos parámetros Θ cuyos datos simulados estén lo suficientemente cerca de los datos reales, es decir, cuando $d(x, x_0) < \epsilon$, donde ϵ es un umbral de tolerancia. Esto define una posterior aproximada:

$$p_{\text{ABC}}(\Theta|x) \propto \int_{d(x, x_0) < \epsilon} p(x|\Theta)p(\Theta) dx \quad (2.2)$$

La intuición es que estamos quedándonos solo con los parámetros que generan simulaciones parecidas a lo observado. Si las estadísticas resumen $s(x)$ son buenas, es decir, capturan

bien la información relevante, y hacemos $\epsilon \rightarrow 0$, entonces esta aproximación converge a la posterior real:

$$p_{\text{ABC}}(\Theta|x) \rightarrow p(\Theta|x)$$

Por otro lado, en la práctica ϵ nunca es cero, así que la posterior aproximada suele ser más ancha que la real. Esto significa que ABC tiende a ser conservador, en el sentido de que introduce más incertidumbre. Otra cosa importante es que la calidad del método depende mucho de la elección de las estadísticas resumen. Si estas pierden información relevante, la inferencia va a ser mala, incluso si ϵ es pequeño. En términos visuales, uno puede pensar que ABC selecciona puntos en el espacio de parámetros que generan simulaciones cercanas a los datos observados. Esto se puede ver como una especie de filtro sobre las simulaciones.

3 Neural Simulation-Based Inference

Aunque los métodos clásicos de inferencia basada en simulación (SBI) han sido bastante exitosos, tienen limitaciones importantes cuando se aplican a problemas con datos complejos o de alta dimensión. Uno de los principales problemas es que dependen de estadísticas resumen $s(x)$ de baja dimensión para comparar simulaciones con datos reales. Esto puede ser problemático porque estas estadísticas no siempre capturan toda la información relevante contenida en los datos originales, lo que puede afectar la calidad de la inferencia. Otro problema importante es la llamada *curse of dimensionality* [23]. A medida que aumenta la dimensión del espacio de parámetros, la cantidad de simulaciones necesarias crece rápidamente, haciendo que estos métodos sean cada vez más ineficientes y costosos computacionalmente.

Estas dificultades han empezado a resolverse gracias a avances en deep learning. Por un lado, las arquitecturas modernas de redes neuronales permiten trabajar directamente con datos complejos, aprendiendo representaciones útiles sin necesidad de diseñar manualmente estadísticas resumen. En lugar de depender de características hechas a mano, el modelo aprende automáticamente qué información es relevante optimizando una función de pérdida. Por otro lado, también se han desarrollado métodos basados en redes neuronales capaces de aproximar distribuciones de probabilidad en espacios de alta dimensión [22], lo que ayuda a mitigar el problema de la dimensionalidad. Esto marca un cambio importante respecto a los métodos clásicos, ya que ahora es posible trabajar de manera más directa con la distribución posterior o con otras cantidades relevantes.

A partir de estos avances, han surgido en los últimos años varios enfoques de SBI basados en redes neuronales. Estos métodos buscan aprovechar la capacidad de las redes para modelar relaciones complejas y aprender distribuciones, ofreciendo alternativas más escalables y flexibles. Su adopción ha crecido bastante en áreas como astrofísica y cosmología, en parte también gracias al desarrollo de herramientas especializadas como `pydelfi` [7] y `sbi` [27]. En general, el teorema de Bayes sugiere distintas formas de abordar el problema usando redes neuronales. Dependiendo de qué parte se quiera aproximar, se pueden definir distintos enfoques: estimar directamente la posterior, aprender la likelihood o aproximar razones entre distribuciones. Estas ideas dan lugar a métodos como Neural Posterior Estimation (NPE) [10, 19], Neural Likelihood Estimation (NLE) [8, 21] y Neural Ratio Estimation (NRE) [5, 11, 14], que se diferencian principalmente en qué objeto probabilístico modelan.

3.1 Neural Posterior Estimation (NPE)

NPE es probablemente el método más directo dentro de neural SBI. La idea es simple: en vez de calcular la posterior de forma explícita, entrenamos una red neuronal para que la aprenda. Es decir, definimos un modelo $q_{\Phi}^{\text{NPE}}(\Theta|x)$ y queremos que se parezca lo más posible a la verdadera posterior:

$$q_{\Phi}^{\text{NPE}}(\Theta|x) \approx p(\Theta|x) \quad (3.1)$$

Para entrenar esto, necesitamos alguna forma de medir qué tan distintas son dos distribuciones. Ahí entra la divergencia KL:

$$D_{KL}(p||q) = \int dx p(x) \ln \frac{p(x)}{q(x)} \quad (3.2)$$

Esta divergencia tiene varias propiedades útiles: no es simétrica, siempre es positiva, y vale cero solo si ambas distribuciones son iguales. En NPE usamos la KL *forward*, o sea $D_{KL}(p||q_{\Phi}^{\text{NPE}})$, porque nos interesa que el modelo cubra bien donde la distribución real tiene masa. La KL en este caso se puede escribir como:

$$D_{KL}(p||q_{\Phi}^{\text{NPE}}) = \int d\Theta p(\Theta|x) \ln \frac{p(\Theta|x)}{q_{\Phi}^{\text{NPE}}(\Theta|x)} = \mathbb{E}_{\Theta \sim p(\Theta|x)} \left[\ln \frac{p(\Theta|x)}{q_{\Phi}^{\text{NPE}}(\Theta|x)} \right] \quad (3.3)$$

El problema es que no podemos muestrear directamente de la posterior $p(\Theta|x)$. Entonces lo que se hace es promediar también sobre datos simulados $x \sim p(x)$, usando la distribución conjunta $p(x, \Theta)$:

$$\mathbb{E}_{x \sim p(x)} [D_{KL}(p||q_{\Phi}^{\text{NPE}})] = \mathbb{E}_{x, \Theta \sim p(x, \Theta)} \left[\ln \frac{p(\Theta|x)}{q_{\Phi}^{\text{NPE}}(\Theta|x)} \right] \quad (3.4)$$

$$= -\mathbb{E}_{x, \Theta \sim p(x, \Theta)} [\ln q_{\Phi}^{\text{NPE}}(\Theta|x)] + \mathbb{E}_{x, \Theta \sim p(x, \Theta)} [\ln p(\Theta|x)] \quad (3.5)$$

El segundo término no depende de la red (de Φ), así que no importa para entrenar. El primero es lo que realmente optimizamos, y a eso se le llama *expected entropy*, aunque no es exactamente la entropía clásica. Al final, la función de pérdida queda:

$$\mathcal{L}_{\text{NPE}} = -\mathbb{E}_{x, \Theta \sim p(x, \Theta)} [\ln q_{\Phi}^{\text{NPE}}(\Theta|x)] \quad (3.6)$$

3.1.1 Estimador de densidad

Para que NPE funcione, tenemos que saber como entrenar una distribución de probabilidad condicional, esto difiere de las tareas clásicas en Machine Learning de regresión o clasificación. En particular, el modelo $q_{\Phi}^{\text{NPE}}(\Theta|x)$ tiene que ser una distribución de probabilidad válida. O sea, tiene que ser no negativa y estar normalizada:

$$\int d\Theta q_{\Phi}^{\text{NPE}}(\Theta|x) = 1 \quad (3.7)$$

Esto no es trivial, así que en la práctica se usan arquitecturas diseñadas específicamente para estimar densidades. Una opción clásica pero desactualizada son las *Mixture Density*

Networks (MDNs) [2], que básicamente hacen que la red neuronal prediga las medias y varianzas de una mezcla de gaussianas. Esto permite modelar distribuciones complejas e incluso multimodales. Hoy en día, lo más común en NPE son los *normalizing flows* [22], porque son más flexibles y permiten evaluar densidades de forma exacta. La idea de los flows es la siguiente: se empieza con una distribución simple y se transforma paso a paso en algo mucho más complejo usando funciones invertibles y diferenciables. Formalmente, se parte de $z_0 \sim p(z_0)$ y se aplica una secuencia de transformaciones:

$$x = z_K = f_K \circ \dots \circ f_1(z_0) \quad (3.8)$$

Esto permite generar muestras fácilmente. Pero lo interesante es que también se puede calcular la densidad exacta del resultado usando cambio de variables:

$$\log p(x) = \log p(z_0) - \sum_{k=1}^K \log |\det J_{f_k}(z_{k-1})| \quad (3.9)$$

La intuición es que cada transformación va deformando el espacio, y el determinante del Jacobiano nos dice cuánto se está expandiendo o contrayendo el volumen en cada paso. Lo bueno de esto es que se pueden construir distribuciones complejas encadenando transformaciones simples. Si se logra parametrizar estas transformaciones con redes neuronales, se obtiene un estimador de densidad muy potente. En este contexto, los parámetros Φ que incluyen tanto los de la distribución base como los de todas las transformaciones, se entrenan con gradiente descendente usando la loss de NPE, lo que permite aprender distribuciones posteriores multimodales.

Hay varias variantes de normalizing flows. Por ejemplo, los *autoregressive flows* (como MAF [20]) hacen que el Jacobiano sea triangular, lo que permite evaluar densidades de forma eficiente, lo que permite un entrenamiento estable. Otros más modernos como *Neural Spline Flows* (NSF) [9] usan transformaciones splines, lo que les da más flexibilidad. Para mas info ver el tutorial de Papamakarios [22].

3.2 Neural Likelihood Estimation (NLE)

Otra forma de hacer neural SBI es en vez de aprender la posterior directamente como en NPE, aprender la likelihood. Es decir, entrenamos un modelo $q_{\Phi}^{\text{NLE}}(x|\Theta)$ que aproxime:

$$q_{\Phi}^{\text{NLE}}(x|\Theta) \approx p(x|\Theta) \quad (3.10)$$

La idea es que en lugar de modelar $p(\Theta|x)$, modelamos $p(x|\Theta)$ y después podemos usar Bayes si queremos recuperar la posterior. El entrenamiento sigue prácticamente la misma lógica que en NPE. Usando muestras del simulador $(x, \Theta) \sim p(x, \Theta)$, la función de pérdida queda como:

$$\mathcal{L}_{\text{NLE}} = -\mathbb{E}_{x, \Theta \sim p(x, \Theta)} [\ln q_{\Phi}^{\text{NLE}}(x|\Theta)] \quad (3.11)$$

O sea, nuevamente es máxima verosimilitud pero ahora sobre la likelihood. Básicamente estamos entrenando la red para que, dado Θ , le asigne alta probabilidad a los datos x que realmente genera el modelo. Al igual que en NPE, necesitamos que $q_{\Phi}^{\text{NLE}}(x|\Theta)$ sea una distribución válida, así que típicamente se usan arquitecturas como normalizing flows para asegurar normalización y flexibilidad.

La elección entre NPE y NLE no es trivial y depende del problema. Se podría pensar en dimensiones: a las redes les resulta más fácil mapear de alta dimensión a baja dimensión que al revés. Entonces, si los datos x son de alta dimensión, podría ser mejor usar NPE, porque se está aprendiendo algo como $x \rightarrow \Theta$. En cambio, si el espacio de parámetros Θ es de alta dimensión, puede ser más conveniente NLE, porque modelar $p(x|\Theta)$ puede ser más manejable. De todas formas en la práctica esto no suele ser tan claro.

3.3 Neural Ratio Estimation (NRE)

Falta por investigar :/ [5, 11, 14]

4 Esquemas de compresión de datos

Aunque los métodos de neural SBI eliminan la necesidad de elegir a mano cosas como el umbral ϵ como en ABC, todavía queda un problema importante: los datos x pueden ser de muy alta dimensión, mientras que los parámetros Θ suelen ser de baja dimensión. Entonces sigue siendo difícil mapear directamente desde x hacia Θ . Igual que en ABC necesitábamos buenas *summary statistics*, acá también necesitamos alguna forma de comprimir los datos. La idea es construir una función $s(x)$ que reduzca la dimensión:

$$x \in \mathbb{R}^{D_x} \rightarrow s(x) \in \mathbb{R}^{D_s}, \quad D_s < D_x \quad (4.1)$$

pero que conserve la mayor cantidad posible de información relevante sobre los parámetros. Idealmente, queremos que trabajar con los datos comprimidos sea casi equivalente a usar los datos originales:

$$p(\Theta|s(x)) \approx p(\Theta|x) \quad (4.2)$$

O sea, que la compresión no destruya información útil para inferencia. En general, hay tres formas principales de abordar esto: usar estadísticas diseñadas a mano basadas en conocimiento del problema, usar métodos que maximizan información como Fisher, o aprender la compresión directamente con redes neuronales junto con el modelo de inferencia.

4.1 Summary Statistics

La forma más simple de reducir la dimensión de los datos es usar *summary statistics* diseñadas a mano antes de aplicar neural SBI. La idea es aprovechar conocimiento del dominio para construir funciones $s(x)$ que capturen lo importante. En algunos casos ideales, estas estadísticas pueden ser suficientes, es decir, contienen toda la información necesaria para inferir los parámetros. Pero en problemas reales, esto rara vez pasa. Entonces hay un trade-off claro: las estadísticas hechas a mano suelen ser interpretables, pero pueden perder información importante si no capturan todo lo relevante.

4.2 Information-Theoretic Compression

Para datos más complejos, hay enfoques que en vez de definir las estadísticas a mano, las aprenden maximizando información. La idea general es encontrar una compresión que

retenga la mayor cantidad posible de información sobre los parámetros, por ejemplo, usando información de Fisher. Un ejemplo es el enfoque de [13], que hace compresión lineal óptima bajo ciertos supuestos y reduce los datos a tantas dimensiones como parámetros haya. Básicamente proyecta los datos en direcciones que maximizan la información. Otro enfoque es el que usan en [1], donde primero se extraen features y luego se comprimen hacia el *score*, que es el gradiente del log-likelihood respecto a los parámetros. Esto permite capturar información relevante incluso en casos no gaussianos. También existen métodos como el de [4], donde directamente se entrena una red neuronal para maximizar el determinante de la matriz de información de Fisher. Acá el trade-off es distinto: se tiene más flexibilidad y potencialmente mejores representaciones, pero se pierde algo de interpretabilidad comparado con estadísticas hechas a mano.

4.3 End-to-End Neural Compression

Una idea aún más moderna es no separar compresión e inferencia, sino entrenarlas juntas. En vez de definir $s(x)$ antes, se usa una red $s_\Psi(x)$ que aprende una representación, y luego otra red q_Φ que hace la inferencia:

$$x \rightarrow s_\Psi(x) \rightarrow q_\Phi(\Theta|x)$$

Ambas redes se entrenan al mismo tiempo usando la misma loss, por ejemplo la de NPE, así que la compresión se adapta automáticamente a lo que necesita la inferencia. Dependiendo del tipo de datos, se pueden usar distintas arquitecturas: CNNs para imágenes, RNNs para secuencias, etc. Este enfoque también se usa en NRE sin problemas. Un detalle importante es que esto no funciona tan bien para NLE: como la likelihood no fuerza a mantener información relevante de los parámetros, el modelo puede colapsar a representaciones poco informativas. Por eso, en NLE suele ser mejor fijar la compresión desde antes. En teoría, este enfoque puede aprender estadísticas casi óptimas, pero en la práctica tiene costos: requiere muchos datos, es menos interpretable, y puede fallar si el modelo se complejiza demasiado con estos pesos y bias extras.

5 Sequential Neural Posterior Estimation

Cuando hacemos SBI, en general tenemos que hacer muchas simulaciones para entrenar el modelo, la elección de prior es importante para que el proceso sea eficiente. Si el prior es muy amplio, la mayoría de las simulaciones van a caer en regiones de baja probabilidad, lo que hace que el entrenamiento sea ineficiente. Por eso, una idea natural es ir adaptando el prior a medida que vamos aprendiendo sobre el posterior, enfocándonos cada vez más en las regiones relevantes. La idea general de los métodos secuenciales de NPE (SNPE) se basan en esto: en vez de muestrear siempre desde el prior $p(\Theta)$, vamos adaptando de dónde sacamos los parámetros. En cada iteración usamos una *proposal* $\tilde{p}(\Theta)$ que típicamente se parece al posterior que llevamos hasta el momento, de modo que concentramos las simulaciones en las regiones que realmente importan para explicar la observación x_0 . Esto hace el proceso mucho más eficiente. Pero acá aparece un problema importante. Si entrenamos el modelo igual que antes, en realidad no estamos aprendiendo el posterior verdadero, sino uno sesgado por la proposal:

$$q_\Phi^{\text{SNPE}}(\Theta | x) \approx \frac{1}{Z} p(x | \Theta) \tilde{p}(\Theta).$$

donde Z es una constante de normalización. Entonces, todos los métodos SNPE básicamente giran en torno a cómo corregir este sesgo introducido por usar una proposal distinta del prior. Ver [10] para una revisión de los distintos métodos SNPE (SNPE-A [19], SNPE-B y SNPE-C o también llamado Automatic Posterior Transformation APT [10]) TAREA PARA LA CASA.

6 Diagnostics and tests

Evaluar si un método de SBI está funcionando bien es clave, porque a diferencia de métodos más clásicos como MCMC, las redes neuronales pueden fallar de forma silenciosa sin que sea obvio. No hay algo como un diagnóstico de convergencia estándar que sea seguro, así que hay que hacer tests específicos para ver si el posterior que aprendimos realmente tiene sentido. La idea general de estos diagnósticos es chequear consistencia estadística usando el simulador, probando el método en muchos datos simulados donde sabemos la verdad.

6.1 Posterior predictive checks (PPC)

Uno de los chequeos más básicos es el *posterior predictive check* (PPC). La idea es simple: se toman muestras de parámetros desde el posterior estimado $p(\Theta | x_0)$, luego simulas nuevos datos usando el modelo $p(x | \Theta)$, y comparas esos datos simulados con los datos observados. Matemáticamente, esto corresponde a mirar la distribución predictiva:

$$p(x | x_0) = \int p(x | \Theta) p(\Theta | x_0) d\Theta.$$

Si los datos simulados se parecen a los observados, es una señal de que el modelo está capturando bien el proceso generativo. Eso sí, pasar el posterior predictive check no garantiza que el posterior sea bueno. Por ejemplo, un posterior muy poco informativo podría generar datos variados que “cubren” los datos observados y aun así no decir nada útil sobre los parámetros. Entonces, este chequeo es necesario, pero no suficiente: sirve como filtro básico, pero hay que complementarlo con otros tests más exigentes.

6.2 Coverage diagnostics

Una forma más exigente de validar los posteriors es a través de *coverage diagnostics*. La idea acá es ver si los intervalos de credibilidad que entrega el modelo realmente contienen el valor verdadero con la frecuencia correcta, en promedio. Para esto se construye un conjunto de calibración generando pares (Θ_i, x_i) desde la distribución conjunta $p(\Theta, x)$, de modo que conocemos el parámetro verdadero que generó cada observación.

Primero, uno de los enfoques mas interpretables es el de *expected coverage* [15]. En este caso se evalúa el posterior completo usando regiones de alta densidad (HPD). La idea es ver si, por ejemplo, una región de credibilidad del 90% realmente contiene el valor verdadero el 90% de las veces. Esto se suele visualizar con gráficos tipo P-P plot o Z-Z plot: si el modelo está bien calibrado, aparece una diagonal. Si el posterior es muy conservador, la curva queda arriba; si es muy confiado, queda abajo. Este método es potente porque evalúa la distribución conjunta y puede detectar problemas en correlaciones entre parámetros. Luego está TARP (*Tests of Accuracy with Random Points*) [17], que usa una idea distinta: construye regiones alrededor

de puntos de referencia aleatorios en el espacio de parámetros y compara distancias entre muestras del posterior y el valor verdadero. En términos simples, verifica si el parámetro real cae lo suficientemente cerca de muestras típicas del posterior. Este enfoque permite detectar ciertos sesgos que métodos basados en regiones HPD pueden pasar por alto, y además es útil cuando no se puede evaluar explícitamente la densidad del modelo. Finalmente, está *Simulation-Based Calibration* (SBC) [26], que es más simple y se basa en un procedimiento de ranking. Para cada ejemplo del conjunto de calibración, se toman varias muestras del posterior y se compara el valor verdadero con ellas, viendo en qué posición queda. Si el posterior está bien calibrado, estos rankings deberían distribuirse uniformemente.

Algo importante es que todos estos tests son computacionalmente caros, porque requieren hacer inferencia muchas veces. Métodos como NPE son más rápidos para esto porque generan muestras directamente, mientras que NLE o NRE suelen necesitar MCMC en cada caso, lo que los hace más lentos. Además, en métodos secuenciales como SNPE, estos tests se complican más porque los posteriors aprendidos dependen de la proposal y no son válidos globalmente. Finalmente, pasar estos tests es una condición necesaria, pero no suficiente. Por ejemplo, un posterior muy ancho puede estar perfectamente calibrado en términos de coverage, pero no ser útil en la práctica. Así que, como siempre, hay que interpretar estos diagnósticos con cuidado.

7 Toy model

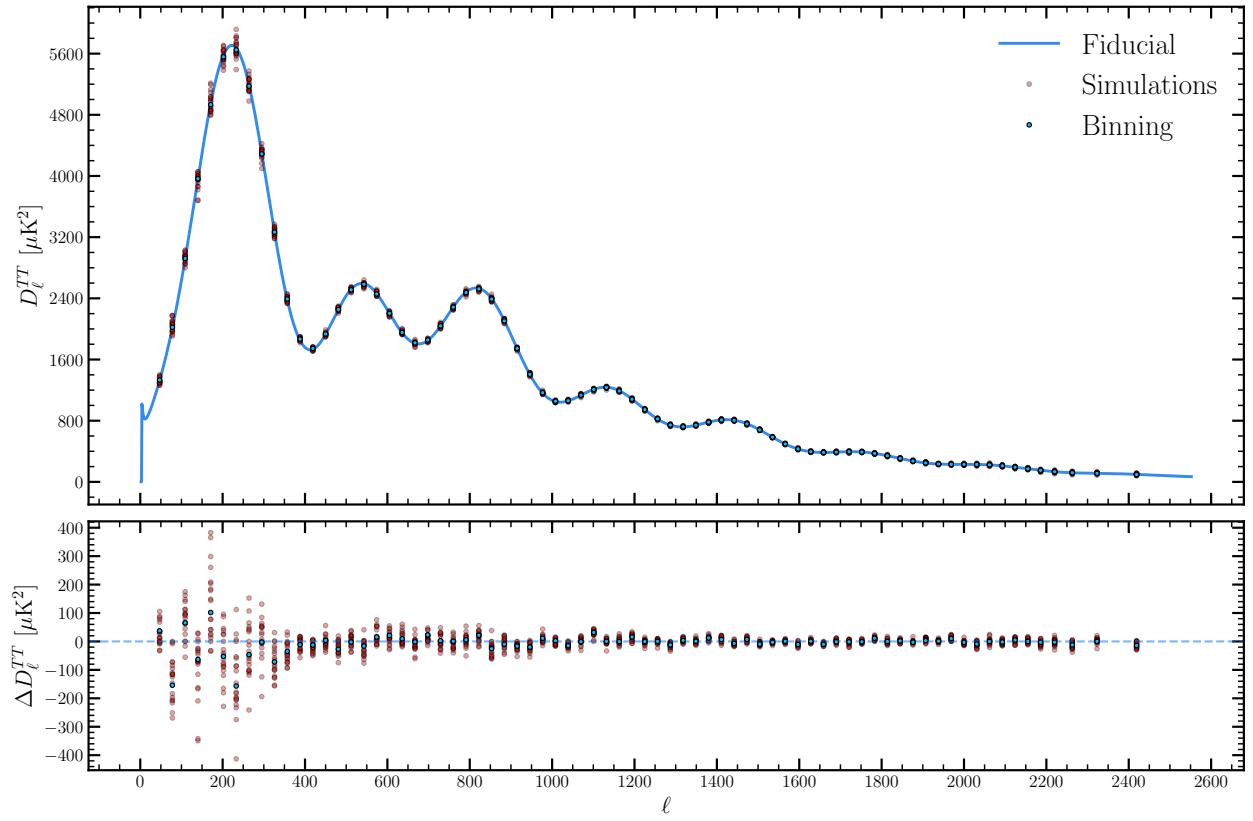


Figure 1: Comparación de simulaciones del modelo toy. Panel superior: espectro de potencia de temperatura D_{ℓ}^{TT} de la simulación fiducial (línea azul) y las simulaciones de observaciones (líneas rojas). Panel inferior: residuales ΔD_{ℓ}^{TT} respecto a la simulación fiducial.

References

- [1] Justin Alsing, Benjamin Wandelt, and Stephen Feeney. Massive optimal data compression and density estimation for scalable, likelihood-free inference in cosmology. *Monthly Notices of the Royal Astronomical Society*, 477(3):2874–2885, March 2018. ISSN 1365-2966. doi: 10.1093/mnras/sty819. URL <http://dx.doi.org/10.1093/mnras/sty819>.
- [2] Christopher M. Bishop. Mixture density networks. 1994. URL <https://api.semanticscholar.org/CorpusID:118227751>.
- [3] Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng. *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, May 2011. ISBN 9780429138508. doi: 10.1201/b10905. URL <http://dx.doi.org/10.1201/b10905>.
- [4] Tom Charnock, Guilhem Lavaux, and Benjamin D. Wandelt. Automatic physical inference with information maximizing neural networks. *Physical Review D*, 97(8), April 2018. ISSN 2470-0029. doi: 10.1103/physrevd.97.083004. URL <http://dx.doi.org/10.1103/PhysRevD.97.083004>.
- [5] Kyle Cranmer, Juan Pavez, and Gilles Louppe. Approximating likelihood ratios with calibrated discriminative classifiers, 2016. URL <https://arxiv.org/abs/1506.02169>.
- [6] Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, May 2020. ISSN 1091-6490. doi: 10.1073/pnas.1912789117. URL <http://dx.doi.org/10.1073/pnas.1912789117>.
- [7] Michael Deistler, Jan Boelts, Peter Steinbach, Guy Moss, Thomas Moreau, Manuel Gloeckler, Pedro L. C. Rodrigues, Julia Linhart, Janne K. Lappalainen, Benjamin Kurt Miller, Pedro J. Gonçalves, Jan-Matthis Lueckmann, Cornelius Schröder, and Jakob H. Macke. Simulation-based inference: A practical guide, 2025. URL <https://arxiv.org/abs/2508.12939>.
- [8] Conor Durkan, George Papamakarios, and Iain Murray. Sequential neural methods for likelihood-free inference, 2018. URL <https://arxiv.org/abs/1811.08723>.
- [9] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows, 2019. URL <https://arxiv.org/abs/1906.04032>.
- [10] David S. Greenberg, Marcel Nonnenmacher, and Jakob H. Macke. Automatic posterior transformation for likelihood-free inference, 2019. URL <https://arxiv.org/abs/1905.07488>.
- [11] Michael U. Gutmann, Ritabrata Dutta, Samuel Kaski, and Jukka Corander. Likelihood-free inference via classification, 2017. URL <https://arxiv.org/abs/1407.4981>.
- [12] W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 04 1970. ISSN 0006-3444. doi: 10.1093/biomet/57.1.97. URL <https://doi.org/10.1093/biomet/57.1.97>.
- [13] A. F. Heavens, R. Jimenez, and O. Lahav. Massive lossless data compression and multiple parameter estimation from galaxy spectra. *Monthly Notices of the Royal Astronomical Society*, 317(4):965–972, October 2000. ISSN 1365-2966. doi: 10.1046/j.1365-8711.2000.03692.x. URL <http://dx.doi.org/10.1046/j.1365-8711.2000.03692.x>.

- [14] Joeri Hermans, Volodimir Begy, and Gilles Louppe. Likelihood-free mcmc with amortized approximate ratio estimators, 2020. URL <https://arxiv.org/abs/1903.04057>.
- [15] Joeri Hermans, Arnaud Delaunoy, François Rozet, Antoine Wehenkel, Volodimir Begy, and Gilles Louppe. A trust crisis in simulation-based inference? your posterior approximations can be unfaithful, 2022. URL <https://arxiv.org/abs/2110.06581>.
- [16] Matt Hoffman, David M. Blei, Chong Wang, and John Paisley. Stochastic variational inference, 2013. URL <https://arxiv.org/abs/1206.7051>.
- [17] Pablo Lemos, Adam Coogan, Yashar Hezaveh, and Laurence Perreault-Levasseur. Sampling-based accuracy testing of posterior estimators for general inference, 2023. URL <https://arxiv.org/abs/2302.03026>.
- [18] Jean-Michel Marin, Pierre Pudlo, Christian P. Robert, and Robin Ryder. Approximate bayesian computational methods, 2011. URL <https://arxiv.org/abs/1101.0955>.
- [19] George Papamakarios and Iain Murray. Fast ϵ -free inference of simulation models with bayesian conditional density estimation, 2018. URL <https://arxiv.org/abs/1605.06376>.
- [20] George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation, 2018. URL <https://arxiv.org/abs/1705.07057>.
- [21] George Papamakarios, David C. Sterratt, and Iain Murray. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows, 2019. URL <https://arxiv.org/abs/1805.07226>.
- [22] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference, 2021. URL <https://arxiv.org/abs/1912.02762>.
- [23] Dennis Prangle. Summary statistics in approximate bayesian computation, 2015. URL <https://arxiv.org/abs/1512.05633>.
- [24] S. A. Sisson, Y. Fan, and M. A. Beaumont. Overview of approximate bayesian computation, 2018. URL <https://arxiv.org/abs/1802.09720>.
- [25] John Skilling. Nested sampling. *AIP Conference Proceedings*, 735(1):395–405, 11 2004. ISSN 0094-243X. doi: 10.1063/1.1835238. URL <https://doi.org/10.1063/1.1835238>.
- [26] Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating bayesian inference algorithms with simulation-based calibration, 2020. URL <https://arxiv.org/abs/1804.06788>.
- [27] Alvaro Tejero-Cantero, Jan Boelts, Michael Deistler, Jan-Matthis Lueckmann, Conor Durkan, Pedro J. Gonçalves, David S. Greenberg, and Jakob H. Macke. sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, 5(52):2505, 2020. doi: 10.21105/joss.02505. URL <https://doi.org/10.21105/joss.02505>.